This guide provides a step-by-step walkthrough for binding an ExpressLRS (ELRS) Transmitter (TX) and Receiver (RX) using the WiFi Web UI and ELRS Configurator. This is particularly useful when versions do not match or when using a standalone module controlled by a microcontroller like an ESP32.

Step 1: Check Firmware Version Compatibility

The most critical rule in ExpressLRS is that the **major version number** (the first digit) must match between your transmitter and receiver. For example, a v4.x.x transmitter cannot communicate with a v3.x.x receiver.

1. **Access the Web UI:** Power on your device and wait for it to enter WiFi mode (usually indicated by a fast double-blink for receivers). Connect your PC to the device's hotspot (e.g., "ExpressLRS RX" and “ExpressLRS TX”) and navigate to 10.0.0.1 in your browser.
2. **Note:**- Wait at least 60 seconds after Starting(turning ON) Both Transmitter and Receiver module in order to make sure both transmitter and receiver are in wifi mode. In wifi Mode the LED light on the receiver module will blink rapidly**. Password to connect to the wifi of Both Tx and Rx is : “expresslrs”**
3. **Verify Versions:** Navigate to the "Information" tab. Check the "Version" field for both your TX and RX.
    - *Example:* If your TX is at **4.0.0** but your RX is at **3.6.0**, they will not bind. You must update the RX.

Step 2: Uploading Firmware to Transmitter and Receiver via ELRS Configurator

When versions do not match, use the ELRS Configurator to build and flash the correct firmware.

1. **Select the Target:** Open the ELRS Configurator and select the exact category and device (e.g., Category: "ZeroDrag 2.4 GHz", Device: "ZeroDrag NexusONE 2.4GHz RX").
2. **Select the Version:** Choose the version that matches your transmitter (e.g., **4.0.0**).
3. **Set the Binding Phrase:** Enter a unique phrase (e.g., Setupdev@123) in the "Binding phrase setup" section. This phrase acts as a digital password.
4. **Build the Firmware:** Click the **BUILD** button to generate a .bin firmware file.

    Note :- **Firmware bin file will be stored in temp folder located somewhere in the local folder of C drive.(e.g “C:\Users\admine\AppData\Local\Temp” or "C:\Users\admine\AppData\Local\Temp\radiomaster.tx_2400.ranger-nano.wifi_6UX0UL\firmware.bin"**
5. **Flash via Web UI:** Go back to your receiver's Web UI (10.0.0.1), navigate to the **Update** tab, select the .bin file you just built(drag or open to select the file), and click **Update**.

Step 3: Binding using the WiFi Web UI

Once the versions match and both devices have the same binding phrase, they will generate an identical **Binding UID**.

1. **Enter the Phrase:** In the Web UI of both devices under the **Options** tab, ensure the "Binding Phrase" field contains your exact phrase.
2. **Save and Reboot:** After saving, both devices will calculate a matching **Binding UID** (e.g., 243, 75, 172, 150, 53, 14).
3. **Successful Link:** When both are powered on normally (not in WiFi mode), the receiver's LED will turn **solid green or blue**, indicating a successful link.

Step 4: Why We Use Code for CRSF and RC Packets

If you are using a standalone module like the RadioMaster Ranger Nano without a standard radio handset, the module acts as a "brain" that requires instructions from an external microcontroller (like an ESP32).

- **Waking Up the TX:** ELRS modules remain in an idle state (often a slow blink) until they receive a valid **CRSF (Crossfire) protocol** signal.
- **RC Packets:** The ESP32 must send a continuous stream of "RC Channels" (control data) at a high speed, such as **400,000 baud**, to tell the TX module it is connected to a "handset" and ready to broadcast.
- **Binding Commands:** A specific "Bind Command" can be sent via code to force the transmitter to search for new receivers without needing a physical button press.

By following these steps, you ensure that your hardware is synchronized, running compatible software, and receiving the necessary data streams to maintain a stable long-range link.

Upload this code to the ESP32 connected to Transmitter in order to bind both Transmitter and Receiver

Important Note :- We don’t need to upload code to the receiver module.

```
#include <HardwareSerial.h>

HardwareSerial CRSFSerial(2);

#define CRSF_TX_PIN 17
```

```
#define CRSF_RX_PIN 16
#define CRSF_BAUDRATE 400000

uint8_t crsf_crc8(const uint8_t *data, uint8_t len) {
 uint8_t crc = 0;
 for (uint8_t i = 0; i < len; i++) {
  crc ^= data[i];
  for (uint8_t j = 0; j < 8; j++) {
   if (crc & 0x80) crc = (crc << 1) ^ 0xD5;
   else crc <<= 1;
  }
 }
 return crc;
}

// Send 16 RC channels at mid-stick (992 = center)
void sendRCChannels() {
 // 16 channels packed into 22 bytes (11-bit each)
 uint8_t payload[22];
 memset(payload, 0, sizeof(payload));

 uint16_t ch = 992; // mid-stick value
 // Pack 16 x 11-bit channels
 uint32_t val = 0;
 int bits = 0;
 int byteIdx = 0;
 for (int i = 0; i < 16; i++) {
  val |= ((uint32_t)ch << bits);
  bits += 11;
  while (bits >= 8) {
   payload[byteIdx++] = val & 0xFF;
```

```
    val >>= 8;
    bits -= 8;
   }
  }

  uint8_t frame[26];
  frame[0] = 0xC8;   // sync
  frame[1] = 24;     // length (22 payload + type + crc)
  frame[2] = 0x16;   // RC channels packed frame type
  memcpy(&frame[3], payload, 22);
  frame[25] = crsf_crc8(&frame[2], 23);

  CRSFSerial.write(frame, 26);
}

void sendBindCommand() {
  uint8_t frame[8];
  frame[0] = 0xEE;
  frame[1] = 0x06;
  frame[2] = 0x32;   // COMMAND
  frame[3] = 0xEA;   // dest: TX
  frame[4] = 0xEF;   // src: handset
  frame[5] = 0x10;   // subcmd RX
  frame[6] = 0x01;   // BIND
  frame[7] = crsf_crc8(&frame[2], 5);

  CRSFSerial.write(frame, 8);
  Serial.println(">>> Bind command sent! Power on RX NOW!");
}

// CRSF parser
```

```
uint8_t rxBuf[64];
uint8_t rxLen = 0;
bool receiving = false;

void readCRSF() {
  while (CRSFSerial.available()) {
    uint8_t b = CRSFSerial.read();
    if (!receiving && (b == 0xC8 || b == 0xEE || b == 0xEA || b == 0xEC)) {
      rxBuf[0] = b; rxLen = 1; receiving = true;
    } else if (receiving) {
      rxBuf[rxLen++] = b;
      if (rxLen == 2 && rxBuf[1] > 60) { receiving = false; rxLen = 0; }
      if (rxLen > 2 && rxLen == rxBuf[1] + 2) {
        uint8_t type = rxBuf[2];
        if (type == 0x1C) {
          Serial.println("*** LINK STATS RECEIVED — RX IS BOUND! ***");
          Serial.print("RSSI: -"); Serial.print(rxBuf[3]); Serial.println(" dBm");
          Serial.print("LQ: ");    Serial.print(rxBuf[5]);  Serial.println("%");
        } else if (type == 0x14) {
          Serial.println("RC Channels OK — TX is transmitting");
        } else {
          Serial.print("Frame type: 0x"); Serial.println(type, HEX);
        }
        receiving = false; rxLen = 0;
      }
    }
  }
}

unsigned long lastRC = 0;
unsigned long lastBind = 0;
```

```
bool bindSent = false;
int bindAttempts = 0;

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("ELRS Bind Sender v2");
  Serial.println("====================");
  Serial.println("Sending RC channels to wake TX...");

  CRSFSerial.begin(CRSF_BAUDRATE, SERIAL_8N1, CRSF_RX_PIN, CRSF_TX_PIN);
  delay(500);
}

void loop() {
  // Send RC channels every 4ms (~250Hz) to keep TX alive
  if (millis() - lastRC >= 4) {
    sendRCChannels();
    lastRC = millis();
  }

  // Send bind command every 3 seconds, up to 10 times
  if (millis() - lastBind >= 3000 && bindAttempts < 10) {
    Serial.print("Bind attempt #"); Serial.println(bindAttempts + 1);
    sendBindCommand();
    lastBind = millis();
    bindAttempts++;
  }

  if (bindAttempts >= 10 && !bindSent) {
    Serial.println("10 attempts done. Type 'b' to retry or check wiring.");
```

```
    bindSent = true;
  }

  // Manual retry
  if (Serial.available()) {
    char c = Serial.read();
    if (c == 'b' || c == 'B') {
      bindAttempts = 0;
      bindSent = false;
      Serial.println("Retrying bind...");
    }
  }

  readCRSF();
}
```